

ID:2403A51062

Name:S.Nithya Shree

Batch03

Date:06-08-2026

```

import pandas as pd
import torch

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv("/content/insurance.csv")

# Display first five rows
print(data.head())

# Check missing values
print(data.isnull().sum())

# Remove missing values
data = data.dropna()

# Separate features and target
X = data[['age']]
y = data[['charges']]

# Normalize
scaler_X = StandardScaler()
scaler_y = StandardScaler()

X = scaler_X.fit_transform(X)
y = scaler_y.fit_transform(y)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Convert to tensors
X_train = torch.FloatTensor(X_train)
X_test = torch.FloatTensor(X_test)

y_train = torch.FloatTensor(y_train)
y_test = torch.FloatTensor(y_test)

print("Data preprocessing completed!")

```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061

4	32	male	28.880	0	no	northwest	3866.85520
age		0					

```
sex          0
bmi          0
children     0
smoker       0
region       0
charges      0
dtype: int64
Data preprocessing completed!
```

```
import torch
import torch.nn as nn

# Build Neural Network
class SalaryModel(nn.Module):
    def __init__(self):
        super(SalaryModel, self).__init__()

        self.fc1 = nn.Linear(1, 16)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(16, 1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

# Create model
model = SalaryModel()

# Loss function
criterion = nn.MSELoss()

# Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

print(model)
```

```
SalaryModel(
  (fc1): Linear(in_features=1, out_features=16, bias=True)
  (relu): ReLU()
  (fc2): Linear(in_features=16, out_features=1, bias=True)
)
```

```
epochs = 100

losses = []

for epoch in range(epochs):

    outputs = model(X_train)

    loss = criterion(outputs, y_train)

    optimizer.zero_grad()
```

```

        loss.backward()

        optimizer.step()

        losses.append(loss.item())

    if (epoch + 1) % 10 == 0:
        print(f"Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}")

```

```

Epoch [10/100], Loss: 0.9471
Epoch [20/100], Loss: 0.9164
Epoch [30/100], Loss: 0.9059
Epoch [40/100], Loss: 0.9068
Epoch [50/100], Loss: 0.9045
Epoch [60/100], Loss: 0.9039
Epoch [70/100], Loss: 0.9039
Epoch [80/100], Loss: 0.9038
Epoch [90/100], Loss: 0.9037
Epoch [100/100], Loss: 0.9037

```

```

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

# Evaluation mode
model.eval()

with torch.no_grad():
    y_pred = model(X_test)

# Convert tensors to NumPy arrays
y_test_np = y_test.numpy()
y_pred_np = y_pred.numpy()

# Calculate metrics
mse = mean_squared_error(y_test_np, y_pred_np)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test_np, y_pred_np)
r2 = r2_score(y_test_np, y_pred_np)

# Print results
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("Mean Absolute Error (MAE):", mae)
print("R2 Score:", r2)

```

```

Mean Squared Error (MSE): 0.9349370002746582
Root Mean Squared Error (RMSE): 0.9669214033594759
Mean Absolute Error (MAE): 0.7598264217376709
R2 Score: 0.11749202013015747

```

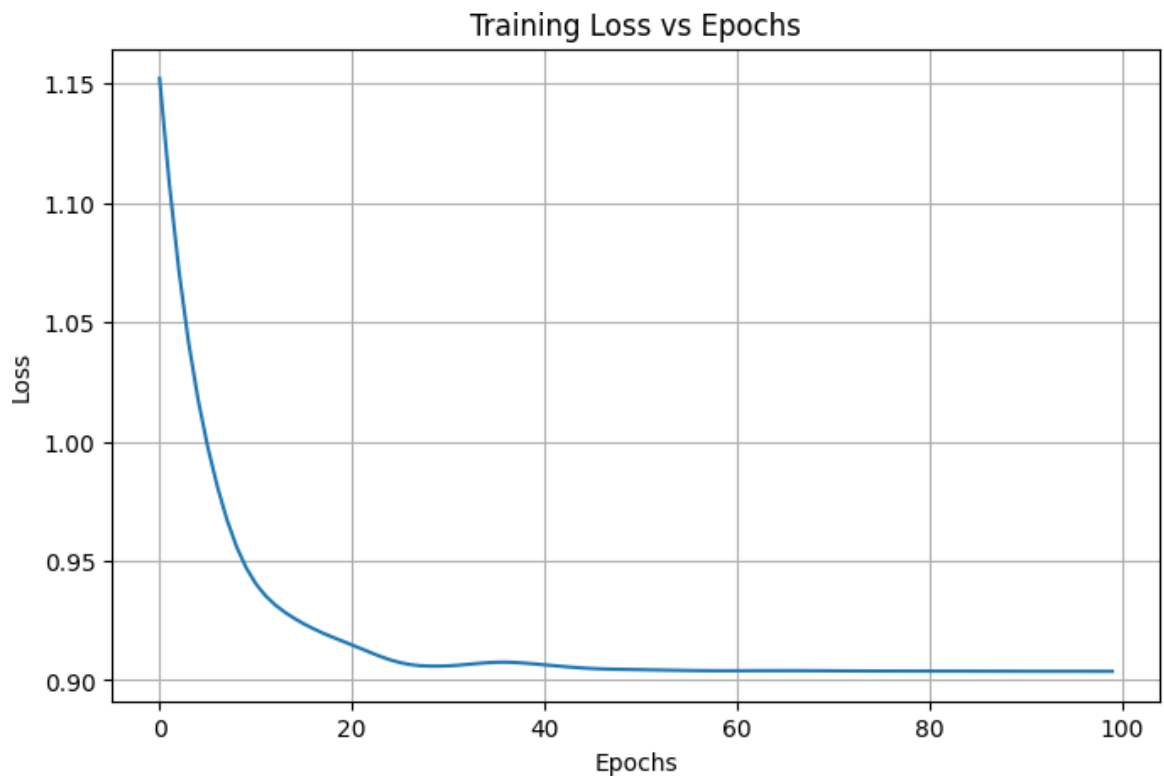
```

import matplotlib.pyplot as plt

# Plot 1: Training Loss
plt.figure(figsize=(8,5))

```

```
plt.plot(losses)
plt.title("Training Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```



```
# Plot 2: Actual vs Predicted
plt.figure(figsize=(8,5))
plt.scatter(y_test_np, y_pred_np)
plt.xlabel("Actual Charges")
plt.ylabel("Predicted Charges")
plt.title("Actual vs Predicted Charges")
plt.grid(True)
plt.show()
```

